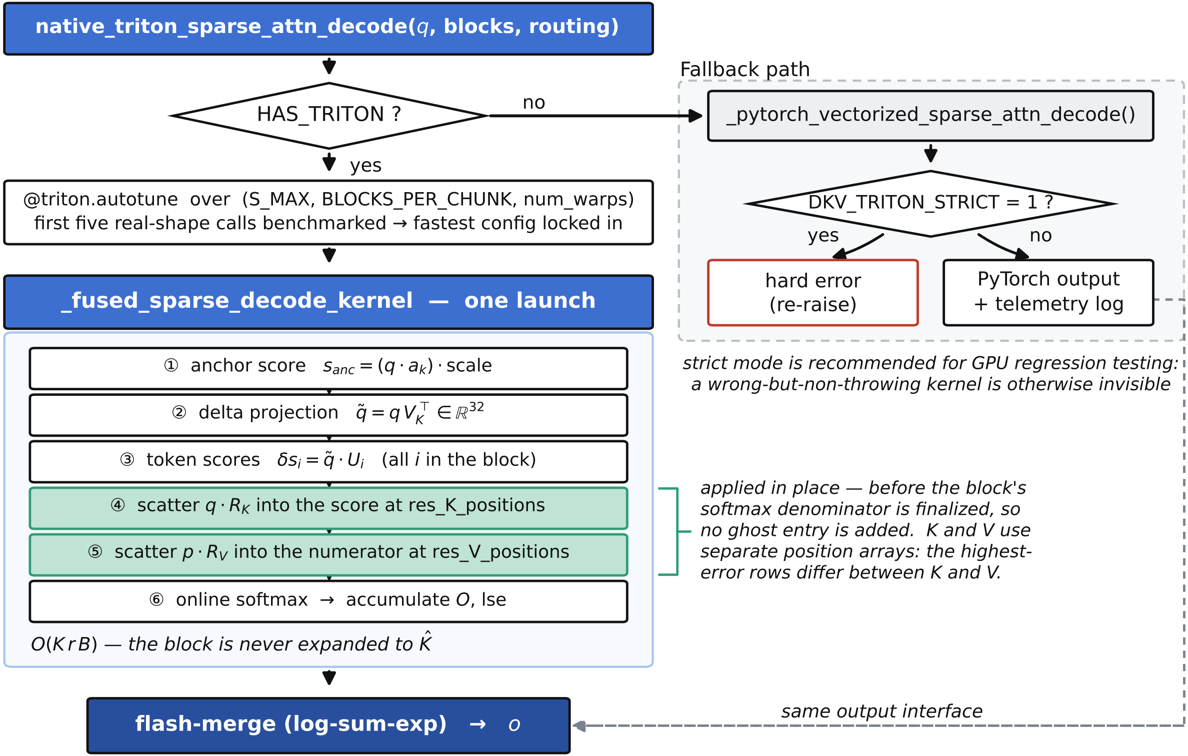

native_triton_sparse_attn_decode(q, blocks, routing)
HAS_TRITON ?
no
yes
@triton.autotune over (S_MAX, BLOCKS_PER_CHUNK, num_warps)
first five real-shape calls benchmarked → fastest config locked in
_fused_sparse_decode_kernel — one launch
① anchor score $s_{anc} = (q \cdot a_k) \cdot \text{scale}$
② delta projection $\tilde{q} = q V_K^\top \in \mathbb{R}^{32}$
③ token scores $\delta s_i = \tilde{q} \cdot U_i$ (all i in the block)
④ scatter $q \cdot R_K$ into the score at res_K_positions
⑤ scatter $p \cdot R_V$ into the numerator at res_V_positions
⑥ online softmax → accumulate O, lse
$O(K r B)$ — the block is never expanded to $\hat{K}$
flash-merge (log-sum-exp) → o
Fallback path
_pytorch_vectorized_sparse_attn_decode()
DKV_TRITON_STRICT = 1 ?
yes
no
hard error
(re-raise)
PyTorch output
+ telemetry log
strict mode is recommended for GPU regression testing:
a wrong-but-non-throwing kernel is otherwise invisible
applied in place — before the block's softmax denominator is finalized, so no ghost entry is added. K and V use separate position arrays: the highest-error rows differ between K and V.
same output interface